Atividade 2
Crítica do artigo “SCX: Stateless KV-Cache Encoding for Cloud-Scale Confidential Transformer Serving” (2025)

Vinicius Trindade Dias Abel
Matrícula: 2020007112

Universidade Federal de Minas Gerais (UFMG)
Belo Horizonte – MG – Brasil

viniciustda@ufmg.br

1. Introdução

Este trabalho é uma análise do artigo *SCX: Stateless KV-Cache Encoding for Cloud-Scale Confidential Transformer Serving*, combinando a leitura do material com os resultados obtidos na execução do código de reprodução.

2. Visão Geral do Trabalho

O artigo apresenta o SCX (*Stateless KV-Cache Encoding*), uma solução inovadora para o problema de privacidade na execuçãode modelos Transformer em ambientes de nuvem. O problema principal que ele tenta resolver é o custo alto das técnicas de privacidade já existentes, como criptografia homomórfica e TEEs (*Trusted Execution Environments*), que inviabilizam o uso em cenários de tempo real.

A ideia central propõe uma abordagem diferente: em vez de proteger todo o modelo ou criptografar todas as operações, o SCX trava apenas o estado intermediário da inferência, o *KV-Cache*, usando chaves controladas pelo usuário. Desta forma, o cache armazenado na nuvem se torna inútil por si só (stateless), impedindo que o provedor de nuvem reconstrua a entrada do usuário ou gere a continuação da resposta de forma independente.

O funcionamento é dividido em três etapas:

I. **Prefill:** O prompt inicial é codificado no dispositivo do usuário e processado de forma interativa com a nuvem, camada por camada, para gerar um KV-Cache inicial codificado.
II. **Keys Sharing:** Para otimizar a performance, o usuário compartilha as chaves das camadas intermediárias do modelo com a nuvem. As chaves da primeira e da última camada, no entanto, permanecem com o usuário, garantindo o controle sobre o início e o fim do processo.
III. **Generation:** Para cada novo token, a comunicação é necessária apenas para as camadas travadas, reduzindo a complexidade da comunicação e, consequentemente, a latência.

O trabalho afirma que essa abordagem atinge uma performance próxima à da inferência em texto plano, sem perda de qualidade e com fortes garantias de privacidade, representando um avanço relevante na área.

3. Análise do Código e Testes

Para complementar a leitura do artigo, executei o código de teste (tests/llama.test.py). Os resultados obtidos confirmam as principais alegações do trabalho:

I. **Equivalência de Saída:** O teste executou uma inferência com o modelo Llama original e, em seguida, com o modelo modificado pelo SCX. A saída do terminal mostra que os dois modelos previram exatamente a mesma sequência de tokens (IDs: 924, 267, 267, 267, 958, 958). Isso valida o Lema 1 (Inference Equivalence) do artigo, provando que o método SCX é, na prática, *lossless* (sem perda de qualidade de predição).

II. **Diferença Mínima nos Logits:** A métrica Max absolute difference between original and SCX last token logits: 0.005859375 é crucial. Ela mede a diferença numérica máxima na saída final do modelo antes da seleção do token. Um valor tão baixo é considerado ruído computacional, por conta de pequenas imprecisões de ponto flutuante, e confirma que a matemática por trás da codificação e decodificação é sólida.

III. **Implementação Funcional:** A execução do teste demonstra que a implementação dos conceitos teóricos é funcional e reproduzível. O código utiliza a biblioteca transformers como base, o que facilita a experimentação.

4. Limitações e Pontos de Discussão

Apesar de ser um trabalho promissor, alguns pontos devem ser observados:

I. **Overhead do prefill (TTFT):** Embora a fase de geração seja muito eficiente, a fase de prefill exige uma comunicação interativa para cada camada do modelo. Para prompts muito longos, esse *Time-To-First-Token* (TTFT) pode ser visivelmente maior em comparação com a inferência em texto plano.

II. **Modelo de Ameaça Semi-Honesto:** O SCX assume um provedor de nuvem semi-honesto (que segue o protocolo, mas tenta inferir o máximo possível). Ele não oferece proteção contra um provedor ativamente malicioso que poderia, por exemplo, tentar ataques de canal lateral (side-channel attacks).

III. **Responsabilidade no Cliente:** O cliente precisa gerar, armazenar e gerenciar as chaves de codificação, o que recai sobre o dispositivo do usuário, adicionando uma camada de complexidade e um potencial ponto de falha.

5. Sugestões de Melhoria

Com base nas limitações apontadas, aqui estão algumas sugestões que podem ser consideradas para fortalecer o SCX:

I. **Redução do overhead no prefill:** Pesquisar formas de paralelizar ou compactar a comunicação durante o prefill, especialmente para prompts longos, pode ajudar a diminuir o *Time-To-First-Token* sem comprometer as garantias de segurança.

II. **Modelos de ameaça mais fortes:** Ampliar a análise de segurança para cenários com adversários ativos, considerando ataques de canal lateral ou tentativas de manipulação do protocolo, reforçaria a aplicabilidade do SCX em ambientes de nuvem hostis.

III. **Gerenciamento de chaves no cliente:** O desenvolvimento de bibliotecas ou ferramentas específicas para lidar com a geração e o armazenamento de chaves poderia reduzir a complexidade para o usuário final, facilitando a integração do SCX em aplicações reais.

IV. **Generalização prática:** Embora o artigo mostre que o design do SCX pode ser aplicado a arquiteturas além do LLaMA, como ViTs e MoEs, seria interessante disponibilizar implementações de referência para diferentes modelos, tornando a técnica mais acessível.

6. Conclusão

O SCX propõe uma solução criativa e eficiente para o problema da privacidade na execução de modelos Transformer em nuvem. A ideia de travar apenas o KV-Cache, em vez de proteger todo o modelo, é prática e mantém a qualidade da inferência sem comprometer o desempenho.

Os resultados obtidos na execução do código confirmam as ideias centrais do artigo: a saída do modelo é equivalente à versão original e as diferenças numéricas são desprezíveis, reforçando a robustez matemática da abordagem.

Ainda assim, existem pontos que merecem atenção. O custo adicional no prefill, o modelo de ameaça restrito a provedores semi-honestos e a responsabilidade maior atribuída ao cliente mostram que o SCX, apesar de promissor, não resolve todos os desafios.

No geral, é uma contribuição relevante. O trabalho abre espaço para pesquisas futuras em direção a técnicas de privacidade mais seguras, leves e práticas, ao mesmo tempo em que oferece uma implementação funcional que comprova a viabilidade da proposta.